

Base Julios GOD

Contents

1	Data structures	2
1.1	Segment tree	2
1.2	Segment tree - Range update, point query	2
1.3	Segment tree - Lazy propagation	2
1.4	Segment tree - Persistence	2
1.5	Fenwick tree	2
1.6	DSU	2
1.7	SQRT decomposition	2
1.8	Trie	2
1.9	Trie XOR	2
1.10	Map custom hash	2
1.11	Ordered set	2
1.12	Merge Sort Tree	2
1.13	Wavelet Tree	2
1.14	Monotonic stack	2
1.15	Matrix operations	2
1.16	Heavy Light Decomposition	2
1.17	Convex Hull Trick	2
1.18	Dynamic Convex Hull Trick	2
2	Graphs	2
2.1	Euler tour	2
2.2	Find bridges	2
2.3	Dijkstra	2
2.4	Binary Lifting LCA	2
2.5	Centroid decomposition	3
2.6	Edmonds karp	3
2.7	Dinics	3
2.8	Bipartite check	3
2.9	Has cycle?	3
2.10	Kuhn - matching	3
2.11	Hopcroft Karp - matching	3
2.12	Min cost - max flow	3
2.13	2sat	3
3	Math	3
3.1	Binpow	3

3.2	Modular inverse	3
3.3	Linear Sieve	3
3.4	Factorials	3
3.5	Prime factorization	3
3.6	Divisors	3
3.7	Identities	3
4	Strings	3
4.1	KMP	3
4.2	Rolling Hash	3
4.3	LCP using hashing + bs	3
4.4	Suffix Automaton	3
4.5	Suffix Array	4
5	Geometry	4
5.1	Convex Hull	4
6	Techniques	4
6.1	MO's algorithm	4
6.2	Parallel binary search	4
6.3	Split objects into light and heavy	4
6.4	Meet in the middle	4
6.5	Dijkstra on ST	4
6.6	Venice set	4
7	Basic techniques	4
7.1	Difference array	4
7.2	Prefix Sum 2D	4
7.3	Random number generator	4
7.4	Coordinate compression	4
7.5	Digit DP	4
7.6	Intersection [L1, R1] and [L2, R2]	4

1 Data structures

1.1 Segment tree

```
1 | #define M ((l + r) >> 1)
```

1.2 Segment tree - Range update, point query

```
1 | // Range update - point query
```

1.3 Segment tree - Lazy propagation

```
1 | #define M ((l + r) >> 1)
```

1.4 Segment tree - Persistence

```
1 | #define M ((l + r) >> 1)
```

1.5 Fenwick tree

```
1 | struct FT{
```

1.6 DSU

```
1 | struct DSU{
```

1.7 SQRT decomposition

```
1 | struct SQ{
```

1.8 Trie

```
1 | struct Node{
```

1.9 Trie XOR

```
1 | struct Node{
```

1.10 Map custom hash

```
1 | #include <ext/pb_ds/assoc_container.hpp>
```

1.11 Ordered set

```
1 | #include<ext/pb_ds/assoc_container.hpp>
```

1.12 Merge Sort Tree

```
1 | #define M ((l + r) >> 1)
```

1.13 Wavelet Tree

```
1 | // indexed in 1
```

1.14 Monotonic stack

```
1 | int main() {
```

1.15 Matrix operations

```
1 | struct Matrix {
```

1.16 Heavy Light Decomposition

```
1 | int sz[N], in[N], out[N], timer, head[N], parent[N];
```

1.17 Convex Hull Trick

```
1 | typedef ll tc;
```

1.18 Dynamic Convex Hull Trick

```
1 | using ll = long long;
```

2 Graphs

2.1 Euler tour

```
1 | vector<int> in(n+1), out(n+1), euler(n+1);
```

2.2 Find bridges

```
1 | int in[N], low[N], timer;
```

2.3 Dijkstra

```
1 | void dijkstra(int start, int n) {
```

2.4 Binary Lifting LCA

```
1 | int depth[N], up[N][LOG];
```

2.5 Centroid decomposition

```
1 | void dfs(int u, int p) {
```

2.6 Edmonds karp

```
1 | const int N = 1e3+5;
```

2.7 Dinics

```
1 | struct Dinic {
```

2.8 Bipartite check

```
1 | int color[N];
```

2.9 Has cycle?

```
1 | int color[N];
```

2.10 Kuhn - matching

```
1 | vector<int> adj[N]; // [0,n)->[0,m]
```

2.11 Hopcroft Karp - matching

```
1 | vector<int> g[MAXN]; // [0,n)->[0,m]
```

2.12 Min cost - max flow

```
1 | typedef ll tf;
```

2.13 2sat

```
1 | struct two_sat {
```

3 Math

3.1 Binpow

```
1 | ll binpow(ll a, ll b, ll m) {
```

3.2 Modular inverse

```
1 | tuple<int, int, int> extendedGCD(int mod, int a) {
```

3.3 Linear Sieve

```
1 | const int N = 1e7+5; // N: range to get primes
```

3.4 Factorials

```
1 | void pre() {
```

3.5 Prime factorization

```
1 | set<int> factors;
```

3.6 Divisors

```
1 | vector<int> div;
```

3.7 Identities

$$C_n = \frac{2(2n-1)}{n+1} C_{n-1}$$

$$C_n = \frac{1}{n+1} \binom{2n}{n}$$

$$C_n \sim \frac{4^n}{n^{3/2} \sqrt{\pi}}$$

$$\sigma(n) = O(\log(\log(n))) \text{ (number of divisors of } n)$$

$$F_{2n+1} = F_n^2 + F_{n+1}^2$$

$$F_{2n} = F_{n+1}^2 - F_{n-1}^2$$

$$\sum_{i=1}^n F_i = F_{n+2} - 1$$

$$F_{n+i} F_{n+j} - F_n F_{n+i+j} = (-1)^n F_i F_j$$

(Fermat's little theorem) $a^p \bmod(p) = a$ $a^{p-1} \bmod(p) = 1$ $a^{p-2} \bmod(p) = a^{-1}$
 (Möbius Inv. Formula) Let $g(n) = \sum_{d|n} f(d)$, then $f(n) = \sum_{d|n} d \mu\left(\frac{n}{d}\right)$.

4 Strings

4.1 KMP

```
1 | vector<int> prefix_function(const string &s){
```

4.2 Rolling Hash

```
1 | #define rep(i,a,b) for(int i=a;i<b;i++)
```

4.3 LCP using hashing + bs

```
1 | int getLCP(int i, int j, int len) {
```

4.4 Suffix Automaton

```
1 | const int M = 27;
```

4.5 Suffix Array

```
1 | int sa[N], lcp[N];
```

5 Geometry

5.1 Convex Hull

```
1 | const double eps = 1e-9;
```

6 Techniques

6.1 MO's algorithm

```
1 | const int N = 3e5+5, B = 550;
```

6.2 Parallel binary search

```
1 | void parallel_bs() {
```

6.3 Split objects into light and heavy

```
1 | vector<pair<int, int>> adj[N];
```

6.4 Meet in the middle

```
1 | int main() {
```

6.5 Dijkstra on ST

```
1 | vector<pair<int, int>> adj[2*4*N];
```

6.6 Venice set

```
1 | struct VeniceSet {
```

7 Basic techniques

7.1 Difference array

```
1 | // If we only need to get the final result, we can increase l and r+1
```

7.2 Prefix Sum 2D

```
1 | // Compute prefix sum 2D
```

7.3 Random number generator

```
1 | // Secure random seed
```

7.4 Coordinate compression

```
1 | // Push to an array all possible values
```

7.5 Digit DP

```
1 | int dp[11][2][2];
```

7.6 Intersection [L1, R1] and [L2, R2]

```
1 | We need to check whether there is any number common to both ranges:
```